

Tamagotchi Wild Edition

Multi-Agent Care for Virtual Pets

Isabella Tagliafico

September 2026

1 Introduction

This project stems from the work developed for the Natural Language Processing course, reinterpreting the same domain (a Wildlife Rescue Center CRAS) using a multi-agent architecture. The facility is divided into four operational areas: Cage Area, Food Storage, Medical Storage, and Treatment Room. Inside it, veterinarians, logistics staff, and feeding personnel must coordinate in real time while respecting precise physical constraints. Each area admits at most two operators simultaneously; the treatment room adds a further constraint, allowing up to three animals on the examination table at once, with each logistics operator limited to transporting a single patient at a time. The system is developed in Python 3.12 with a distributed multi-agent architecture based on SPADE (Smart Python Agent Development Environment, v4.1.4), which manages the agent lifecycle and asynchronous communication over the XMPP network. The deliberative component of the operators is implemented through the SPADE-BDI extension (v0.3.2), which realizes the Belief-Desire-Intention cognitive model via the AgentSpeak interpreter. Real-time monitoring is handled by NiceGUI (v3.15.0), which generates a web dashboard accessible from the browser.

2 Incremental Development Methodology and System Implementation

The development approach of the project was incremental, as there were multiple architectural layers to address.

2.1 Phase 0: Compatibility Study

Initially, lacking certainty that the chosen libraries (Python 3.12, SPADE for the XMPP network, and the BDI interpreter for AgentSpeak) were fully compatible with each other, it was decided to build a disposable minimal prototype with two agents to verify their integration. A command-line smoke test was then executed to confirm that the local XMPP server started and responded correctly.

2.2 Phase 1: Foundation of Domain and Environment

Before introducing any agent, the physical model of the world was implemented, defining what actions are possible and under what conditions. The following components were built:

- the 12×8 grid with the geometric subdivision into four areas (Food Storage, Medical Storage, Cage Area, Treatment Room);
- the base entities (`Position`, `Animal`, `Cage`, `Bowl`, food stocks, medicine stocks) and the `EnvironmentState` controller.

This phase was fundamental because it allowed building the physical and mathematical foundations of the world in pure Python, without yet introducing SPADE, the XMPP network, or BDI logic. In this phase, the following modules were written:

- `domain/entities.py`, where the base data structures were defined:
 - `Position(x, y)`: integer coordinates on the grid;
 - `Area`: rooms with their cell perimeters and respective capacities;
 - `Animal`, `Cage`, `Bowl`: physical objects with unique identifiers;
 - `HealthStatus`: finite health states (`HEALTHY`, `HUNGRY`, `SICK`, `IN_TREATMENT`, `TREATED`, `IN_OUTBOUND_TRANSPORT`, `IN_RETURN_TRANSPORT`, `READY_FOR_TRANSPORT`).
- `environment/state.py`: The `EnvironmentState` class was created to model the physical space and the laws of the world. It maintains the ground truth of all entities: the positions of animals, coordinates of obstacles, bowl levels, and quantities of food and medicine stocks. Furthermore, it validates physical actions, verifying whether an action attempted by an operator is permitted according to the world rules. It is a passive component: it makes no autonomous decisions and possesses no goals, beliefs, or desires; in summary, it represents the simulated physical environment (the ground truth).

Unit tests were written to validate each world rule before moving on.

2.3 Phase 2: Agent Creation and Initial Workflow (Feeding)

In this phase, the first real SPADE agents were created, connected to the XMPP network and guided by BDI logical plans. Following the incremental methodology, it was decided not to implement the complete system immediately, but to validate execution starting from a single hungry animal with a single empty bowl. Notably, agents represent the staff operators of the center, whereas animals are not agents but passive physical objects. In this phase, the following entities were created:

- 1 `EnvironmentAgent` (the centralized environment arbiter);
- 1 `LogisticsAgent` (the coordinator);
- 1 `FeedingAgent` (the operational executor);
- 1 cage with 1 empty bowl and the storage room with food stocks.

To initially verify baseline operations, the following aspects were temporarily excluded:

- the presence of veterinarians and medical treatments;
- the presence of duplicate operators (only a single agent per role);
- room capacity constraints (no waiting queues at room entrances);
- the graphical interface (operating solely via terminal logs).

What was implemented:

1. **The initial BDI plans (.asl):**

- `logistics.asl`: receives the `bowl_empty` belief and adopts the plan to assign it to the feeding operator;
- `feeding.asl`: receives the `feeding_task` belief and triggers the `complete_feeding` operational plan.

2. **The FIPA message chain via XMPP:**

- The `EnvironmentAgent` sends a perception message to the `LogisticsAgent` informing it that the bowl in the cage is empty;
- The `LogisticsAgent` sends a request to the `FeedingAgent` specifying the task to be performed;
- The `FeedingAgent` accepts the task: it requests permission from the environment to withdraw food from the stocks (which are decremented from the total) and subsequently requests the action to refill the bowl, whose level is updated to 1 by the environment to record full availability;
- The `FeedingAgent` finally notifies the `LogisticsAgent` of the successful completion of the task.

Dedicated tests were also created in this phase to verify that all operations functioned correctly.

2.4 Phase 3: Medical Treatment and Transport

Phase 3 introduced higher complexity, involving the initial forms of cooperation between heterogeneous agents. An animal cannot move autonomously from the cage area to the treatment room; therefore, the intervention of an operator was required for transportation. Furthermore, for the administration of medical treatments, it was essential to introduce the Veterinary agent role. What was implemented in Phase 3:

- **The VeterinaryAgent and its BDI plan (`veterinary.asl`):** introduces the specialized medical agent, which receives notifications of sick animals, requests transport from logistics, retrieves medicine from medical storage, and administers treatment in the treatment room.
- **The transport role of the LogisticsAgent:** the logistics operator takes on the task of transporting the animal from the Cage Area to the Treatment Room and vice versa. It is triggered directly by the veterinary agent for both outbound and return transport.
- **The animal finite state machine:** the animal does not heal instantaneously, but traverses a strict sequence of 6 physical states verified by the `EnvironmentAgent`:
 - **SICK:** sick inside its cage (it cannot be treated there);
 - **IN_OUTBOUND_TRANSPORT:** picked up by the logistics operator and traveling toward the treatment room (no other agent can interact with it);
 - **IN_TREATMENT:** placed on the examination table in the treatment room awaiting the doctor;
 - **TREATED:** treated with the drug by the veterinarian, but still on the table;
 - **IN_RETURN_TRANSPORT:** picked up again by the logistics operator for the return trip;
 - **HEALTHY:** returned to its cage of origin, healed and free.
- **Physical clinical constraints:**
 - table capacity: a maximum of 3 patients simultaneously in the treatment room;
 - carrying capacity: each logistics operator can carry only one animal at a time;
 - medication precondition: the veterinarian cannot perform the treatment without first retrieving the dose from the Medical Storage.

Specific tests were also executed for this phase to ensure correct behavioral execution.

2.5 Phases 4 and 5: Scalability and Concurrency

This part addressed the final and most complex challenge: scaling the number of animals and operators while managing concurrency. In earlier phases (Phases 2 and 3), management was simplified by having a single agent per role. With multiple agents, fundamental logistical issues emerged:

- **Task contention (*Race Condition*)**: two operators attempting to perform the same task simultaneously (e.g., refilling the same bowl).
- **Physical bottleneck (*Overcrowding and Deadlock*)**: operators attempting to enter the same room at the same time, blocking each other.

In **Phase 4**, runtime configuration was implemented to select the number of animals (up to a maximum of 40) and operators (up to a maximum of 15). Dynamic world generation was also introduced: launching 5 animals, for instance, automatically instantiates 5 cages, 5 distinct animal species, and 10 independent tasks. Finally, the Round-Robin algorithm was adopted to distribute tasks evenly among all available operators (e.g., with 3 feeding operators and 6 bowls, each operator takes charge of 2). **Phase 5** introduced concurrency management and mutual exclusion:

- **Atomic task reservation (CLAIM_TASK)**: prior to initiating any action, the operator must register the task with the `EnvironmentAgent`. The first agent to arrive acquires the lock; if a second agent requests the same task, the environment rejects it, signaling that the work has already been assigned to another operator.
- **Room semaphores (ACQUIRE_AREA and RELEASE_AREA)**: each room (Food Storage, Medical Storage, Treatment Room, Cage Area) admits a maximum of 2 occupants simultaneously.
- **Asynchronous retry loop (*Deadlock Prevention*)**: if an operator requests access to an area where 2 colleagues are already present, the environment replies with `accepted=False`. The operator does not block or terminate execution: it enters a short asynchronous pause (`await asyncio.sleep(0.05)`) and retries requesting entry until one of the two occupants exits and issues `RELEASE_AREA`.

Dedicated tests were also established in this phase to verify that the concurrent workflow operated without anomalies.

2.6 Phase 6: Graphical User Interface Implementation

In a distributed multi-agent system communicating via XMPP, tracking the progression of actions solely through terminal log lines proved impractical. A graphical user interface was therefore developed to make the simulation observable in real time; the frontend code was built with the aid of generative AI tools. The `VisualizationAgent` was therefore created, acting as a passive

camera that receives state snapshots from the environment. The bridge module `gui_bridge.py` was then implemented to transmit data to the NiceGUI web server, which renders the interactive SVG map in real time, displaying operator movements and status updates dynamically.

3 Domain Modeling and Physical Environment

This section formalizes the spatial layout and the physical rules governing the rescue center. The domain model is implemented in pure Python, completely decoupled from network messaging (SPADE/XMPP), BDI reasoning, and graphical rendering.

3.1 Domain Entities (`domain/entities.py`)

All physical elements of the center are modeled using immutable classes (`@dataclass(frozen=True)`), in order to prevent accidental modifications and ensure safety among concurrent tasks:

- **Space and Rooms:** The world is a grid of coordinates `Position(x, y)` of size 12×8 , divided into four closed areas (**Area**): *Food Storage*, *Medical Storage*, *Cage Area*, and *Treatment Room*.
- **Objects and Resources:** Cages (`Cage`) and bowls (`Bowl`) are located at fixed points on the map. Stocks of food (`FoodStock`) and medicines (`MedicineStock`) keep track of the numerical quantities remaining in the storages.
- **Animal Life Cycle (`HealthStatus`):** The domain enumeration defines eight states (including `HUNGRY` for feeding and `READY_FOR_TRANSPORT` for pre-transit staging), but the medical recovery cycle enforced by the environment follows a mandatory sequence of six physical states:

$$\begin{aligned} \text{SICK} &\rightarrow \text{IN_OUTBOUND_TRANSPORT} \rightarrow \text{IN_TREATMENT} \\ &\rightarrow \text{TREATED} \rightarrow \text{IN_RETURN_TRANSPORT} \rightarrow \text{HEALTHY} \end{aligned}$$

3.2 Actions and Contracts (`domain/actions.py`)

Agents cannot directly touch the world data: to do anything they must send a formal request:

- **Action Types (`ActionType`):** An enumeration of 15 permitted domain operations, including reserving tasks (`CLAIM_TASK`), entering or leaving rooms (`ACQUIRE_AREA`, `RELEASE_AREA`), managing resources (`TAKE_FOOD`, `TAKE_MEDICINE`, `FILL_BOWL`), and executing medical stages (`PICKUP_SICK_ANIMAL`, `DELIVER_TO_TREATMENT`, `TREAT_ANIMAL`, `PICKUP_TREATED_ANIMAL`, `RETURN_ANIMAL_TO_CAGE`).

- **Commands and Outcomes** (`ActionCommand` and `ActionResult`): For each requested action, the environment responds atomically stating whether it was accepted (`accepted = True`) or rejected (`accepted = False`), always specifying the reason for rejection.

3.3 Authoritative Environment (`environment/state.py`)

The `EnvironmentState` class is the only true custodian of the state of the world (*ground truth*). It does not reason and makes no decisions: it only checks that physical rules are respected before executing any modification:

- **Room Capacity Rule:** In each room there can be at most two operators at the same time. If a room is already full, the environment rejects entry and the agent waits outside until a spot is freed.
- **Clinical and Transport Rules:** On the examination table of the treatment room there can be at most three animals simultaneously, and each logistics operator can transport only one animal at a time.
- **History and Snapshots:** At each valid action, the environment assigns a progressive number to the event and creates an exact, protected copy of the state (`WorldSnapshot`), which is then sent to the graphical screen without risking altering the simulation data.

4 Multi-Agent System Architecture

Within the project, a total of five distinct types of agents operate. They are not all built the same way: three of them (`VeterinaryAgent`, `LogisticsAgent`, `FeedingAgent`) are cognitive BDI agents : they maintain an internal belief base, adopt goals, and deliberate through `AgentSpeak` plans before acting. The remaining two (`EnvironmentAgent` and `VisualizationAgent`) are purely reactive: they hold no beliefs and make no decisions, but simply respond to incoming messages according to fixed rules.

- `EnvironmentAgent` — reactive, no BDI
- `LogisticsAgent` — BDI cognitive operator
- `FeedingAgent` — BDI cognitive operator
- `VeterinaryAgent` — BDI cognitive operator
- `VisualizationAgent` — reactive, no BDI

4.1 EnvironmentAgent

- **Role:** Acts as the XMPP communication bridge between the physical world model (`EnvironmentState`) and the multi-agent network.

- **Architecture:** It is a reactive SPADE agent implemented in pure Python. Unlike the other agents, it does not employ the BDI paradigm: its behavior is purely deterministic and reactive, limited to receiving requests and verifying their compliance against the physical rules of the world.

4.1.1 Operational Example

- If an operator asks: *"Can I take a dose of medicine?"*, the Environment does not engage in complex deliberation: it deterministically checks whether the remaining stock is greater than zero. If so, it responds positively and decrements the stock by one dose; otherwise, it rejects the request.
- If an operator asks: *"Can I enter the treatment room?"*, the Environment checks whether the current number of occupants is below maximum capacity: if so, it approves access; otherwise, it rejects it.

4.1.2 Core Responsibilities

- Receives physical action requests sent by operators (`ActionRequest`) and responds by accepting or rejecting them in compliance with the laws of the world;
- Acts as a sensor for the facility: monitors the state of the center and sends perception notifications to designated agents according to the FIPA standard (e.g., notifying Logistics of an empty bowl or the Veterinarian of a sick animal);
- Transmits state update snapshots to the `VisualizationAgent`;
- Ensures simulation integrity, preventing the execution of any unlawful or physically impossible action.

4.1.3 Physical Request Processing

The management logic for physical actions is defined within the module `src/tamagotchi_wild/agents/environment_agent.py`:

Listing 1: Physical action request processing in `environment_agent.py`

```
def process_action_request(world: EnvironmentState, body: str) ->
    ActionResponse:
    try:
        request = ActionRequest.from_json(body)
    except MessageContractError as exc:
        return ActionResponse(
            task_id="unknown",
            accepted=False,
            reason=str(exc),
            event_sequence=None,
        )
    return ActionResponse.from_result(
        request.task_id, world.apply(request.to_command())
```

)

This function illustrates the primary workflow: the agent receives the message body, deserializes it into a structured request object, and forwards it to the physical environment (`world.apply`), returning a response that specifies whether the action was accepted or rejected with the corresponding reason.

4.1.4 Reception Cycle and FIPA-ACL Interaction

The asynchronous cycle that receives and manages requests is implemented via SPADE's cyclic behavior (`RequestBehaviour`):

Listing 2: Asynchronous FIPA-ACL request handling in `RequestBehaviour`

```
class RequestBehaviour(CyclicBehaviour):
    async def run(self) -> None:
        message = await self.receive(timeout=1)
        if message is None:
            return
        request = None
        try:
            request = ActionRequest.from_json(message.body)
        except MessageContractError:
            pass
        response_data = process_action_request(self.agent.world,
            message.body)
        response = Message(to=str(message.sender))
        conversation_id = message.get_metadata("conversation-id")
        response.thread = message.thread or conversation_id
        response.set_metadata(
            "performative", "inform" if response_data.accepted else
            "failure"
        )
        response.set_metadata("ontology", ENVIRONMENT_ONTOLOGY)
        response.set_metadata("language", MESSAGE_LANGUAGE)
        if conversation_id:
            response.set_metadata("conversation-id",
                conversation_id)
        response.body = response_data.to_json()
        await self.send(response)
```

This behavior constitutes the asynchronous cycle that processes operator requests and formalizes communication using the **FIPA-ACL** standard (Foundation for Intelligent Physical Agents - Agent Communication Language), which regulates the format and intentionality of inter-agent messages. The FIPA standard specifies various communicative performatives (including `request`, `inform`, `failure`, and `agree`). In the responses of the `EnvironmentAgent`, only `inform` and `failure` labels are used:

- **inform**: notifies the agent of the successful application of the action in the real world.
- **failure**: notifies the agent of the rejection of the attempted action (for instance, when requested resources are exhausted or room access is saturated).

The `EnvironmentAgent` does not use the `agree` performative. An `agree` message denotes formal acceptance of a task requiring deferred execution over time, as observed in operational agents like the `FeedingAgent`. In contrast, the environment validates and executes actions atomically and instantaneously, making an intermediate execution promise redundant.

4.1.5 Sensory Perception Triggers

In addition to passive request handling, the `EnvironmentAgent` acts proactively as a sensor for the facility through two methods dedicated to signaling hunger and animal illness:

Listing 3: Perception publishing methods in `EnvironmentAgent`

```

async def publish_bowl_empty(
    self,
    recipient: str,
    task_id: str,
    cage_id: str,
    bowl_id: str,
    timeout_seconds: float = 5.0,
) -> None:
    sent = asyncio.Event()
    self.add_behaviour(
        self.PerceptionSender(
            recipient,
            BowlEmptyPerception(task_id, cage_id, bowl_id),
            sent,
        )
    )
    await asyncio.wait_for(sent.wait(), timeout=timeout_seconds)
async def publish_sick_animal(
    self,
    recipient: str,
    task_id: str,
    animal_id: str,
    cage_id: str,
    timeout_seconds: float = 5.0,
) -> None:
    sent = asyncio.Event()
    self.add_behaviour(
        self.SickAnimalSender(
            recipient,
            SickAnimalPerception(task_id, animal_id, cage_id),
            sent,
        )
    )
    await asyncio.wait_for(sent.wait(), timeout=timeout_seconds)

```

These two methods serve as sensory alarms, signaling respectively an empty bowl condition and the detection of a sick animal. From a technical perspective, both methods share the same operational mechanism:

1. Instantiate and attach a temporary, disposable SPADE behavior (`OneShotBehaviour`, namely `PerceptionSender` and `SickAnimalSender`);

2. Serialize the details of the perceptual event into JSON format within an XMPP message carrying the FIPA performative **inform**;
3. Transmit the message to the designated recipient across the XMPP network;
4. Via the instruction `await asyncio.wait_for(sent.wait())`, suspend execution pending confirmation of successful message transmission before proceeding.

4.2 FeedingAgent

4.2.1 Role and Nature of the Agent

The **FeedingAgent** is the operational staff member responsible for the sustenance of hospitalized animals. Its fundamental purpose is to refill empty bowls while ensuring resource consistency and preventing operational conflicts with other colleagues. Unlike the **EnvironmentAgent** (which is a reactive controller without deliberative capabilities), the **FeedingAgent** adopts a two-tier hybrid architecture:

- The high-level deliberative tier models reasoning and mental states via the BDI architecture written in AgentSpeak(L);
- The low-level executive tier manages distributed communication, asynchronous actions, and physical interaction in the world via Python and the SPADE platform.

4.2.2 Interlocutors

The agent constantly communicates with only two entities in the system:

- **With the LogisticsAgent:**
 - Receives task assignments via FIPA messages with the **request** performative (*"Refill the bowl in cage X"*);
 - Responds by sending a FIPA **agree** message (to confirm the start of work), **refuse** (if the task has already been taken by another operator), or **inform** (to notify that the bowl has been successfully refilled).
- **With the EnvironmentAgent:**
 - Communicates via the `cras.environment` ontology to validate physical actions: requests exclusive task reservation, room access and release, food retrieval from storage, and pouring into the bowl.

4.2.3 Complete Operational Cycle

The life cycle of each feeding task is carried out in six linear steps:

1. **Reception:** the agent intercepts the service order from Logistics containing the cage ID and bowl ID.
2. **Atomic Task Reservation:** before moving, it contacts the Environment to acquire the lock on the task. If a colleague has already taken charge of the same bowl, it responds to the `LogisticsAgent` with `refuse`. If it acquires the lock, it starts the plan.
3. **Acceptance:** sends a formal `agree` message to the `LogisticsAgent` to formally commit to executing the task.
4. **Stock Retrieval and Verification:** moves to the Food Storage, enters respecting the capacity limit, and attempts the `TAKE_FOOD` action. The Environment verifies stock: if the quantity is sufficient, it decrements the storage; otherwise, the action fails due to resource exhaustion and the task is aborted.
5. **Bowl Refilling:** reaches the assigned cage in the Cage Area and executes the `FILL_BOWL` action.
6. **Closure:** sends the final FIPA `inform` message to Logistics to declare task completion and updates its internal belief base.

4.2.4 Tier Separation: Beliefs File (.asl) vs. Python File (.py)

To avoid confusion between the theoretical plan and practical execution, the agent is clearly separated into two files:

The Beliefs File: `src/tamagotchi_wild/bdi/feeding.asl` This component encapsulates the declarative deliberative layer of the agent. It performs no numerical computation or network I/O; its role is confined to managing the belief base, adopting goals, and executing plans.

```
!boot.
+!boot <-
    .mark_bdi_ready.
+feeding_task(TaskId, CageId, BowlId) <-
    !complete_feeding(TaskId, CageId, BowlId).
+!complete_feeding(TaskId, CageId, BowlId) <-
    .execute_feeding(TaskId, CageId, BowlId).
+feeding_succeeded(TaskId) <-
    .record_feeding_outcome(TaskId, completed).
+feeding_failed(TaskId) <-
    .record_feeding_outcome(TaskId, failed).
```

- **Initialization:** the `!boot` goal starts the agent and notifies the system that the logical engine is ready (`.mark_bdi_ready`);

- **Reaction to event:** acquiring the `+feeding_task` belief triggers the `!complete_feeding` plan, which delegates the physical action to the Python module by calling `.execute_feeding`;
- **Conclusion:** the returning beliefs `+feeding_succeeded` or `+feeding_failed` formally record the final outcome in the agent's mind.

The Python Executive File: `feeding_agent.py` The `feeding_agent.py` module constitutes solely the executive component of the agent: it makes no autonomous decisions, but translates what the BDI plan has established into physical actions and network messages. The material execution is encapsulated in the `_execute` method of the `ExecuteFeeding` class:

```
class ExecuteFeeding(CyclicBehaviour):
    async def _execute(self) -> None:
        requester = self.agent.task_requesters[self.task_id]
        await self._send_status(requester, "agree", "accepted")
        self.agent.activity_log.record(
            self.task_id,
            self.agent.agent_label,
            "task_accepted",
        )
        take_result = await self._perform_in_area(
            self.agent.food_area_id,
            self.agent.food_position,
            ActionType.TAKE_FOOD,
            self.agent.food_stock_id,
            1,
        )
        if not take_result.accepted:
            await environment_action(
                self,
                self.task_id,
                ActionType.FAIL_TASK,
                self.task_id,
            )
            await self._fail(requester, take_result.reason)
            return
        fill_result = await self._perform_in_area(
            self.agent.cage_area_id,
            self.agent.bowl_position,
            ActionType.FILL_BOWL,
            self.bowl_id,
            1,
        )
        if not fill_result.accepted:
            await self._fail(requester, fill_result.reason)
            return
        await self._send_status(requester, "inform", "completed")
        self.agent.activity_log.record(
            self.task_id,
            self.agent.agent_label,
            "task_completed",
        )
```

```
self.agent.bdi.set_belief("feeding_succeeded", self.task_id
)
```

Unlike the `EnvironmentAgent` (which applied instantaneous actions and responded solely with `inform` or `failure`), this code clearly illustrates the sequential adoption of two distinct FIPA performatives, necessary for a task that develops over time:

- **agree**: sent immediately to the `LogisticsAgent` to confirm formal commitment to the task;
- **inform**: sent at the end to certify that the bowl is full, closing the cycle with the update of the internal belief.

4.3 LogisticsAgent

4.3.1 Role and Nature of the Agent

The `LogisticsAgent` is the operational coordinator of the facility and the physical carrier responsible for animal transportation. Its purpose is twofold: to balance and dispatch feeding tasks among available operators and to manage the transport of patients between the cage area and the treatment room. Like the `FeedingAgent`, it adopts a two-tier hybrid architecture combining BDI logic in AgentSpeak for decision-making and SPADE in Python for networking:

- While the `FeedingAgent` is a pure operational executor, the `LogisticsAgent` combines both coordination (assigning food tasks to others) and direct physical action (personally transporting animals).

4.3.2 Interlocutors

The agent communicates with three entities:

- **With the `EnvironmentAgent`:**
 - Receives sensor notifications regarding empty bowls;
 - Validates physical actions: requests task reservation, animal pickup from the cage, delivery to the treatment room, and return maneuvers.
- **With the `FeedingAgent`:**
 - Dispatches replenishment orders via FIPA messages with the **request** performative;
 - Receives **agree**, **refuse**, or **inform** replies from the feeding operator.
- **With the `VeterinaryAgent`:**
 - Receives transport requests for outbound transit to the treatment room or return to the cage;
 - Responds confirming commitment via **agree**, signaling possible failures with **failure**, or notifying completed delivery with **inform**.

4.3.3 Complete Operational Cycle

The agent manages two operational workflows:

Workflow A: Feeding Coordination

1. **Reception:** receives the empty bowl alert from the Environment;
2. **Dispatching:** uses the Round-Robin algorithm to select an available feeding operator and evenly distribute the workload;
3. **Assignment:** sends the request to the operator, waits for acceptance, and receives the final completion notification.

Workflow B: Medical Transport of Animals

1. **Order Reception:** the Veterinarian requests transport of a patient for outbound or return transit;
2. **Atomic Task Reservation:** contacts the Environment to lock the task and prevent colleagues from overlapping;
3. **Acceptance:** sends an **agree** message to the Veterinarian to confirm the start of transit;
4. **Animal Pickup:** reaches the starting location and takes custody of the animal (only one at a time);
5. **Delivery:** transports the animal to its destination (on the examination table or in the origin cage after treatment);
6. **Closure:** sends the final **inform** message to the Veterinarian to certify completed delivery.

4.3.4 Tier Separation: Beliefs File (.asl) vs. Python File (.py)

The agent is clearly separated between logical mind and material execution:

The Beliefs File: `src/tamagotchi_wild/bdi/logistics.asl` Unlike the **FeedingAgent** (which holds a single belief for a task it executes entirely on its own), the BDI mind of the **LogisticsAgent** manages two profoundly different types of beliefs:

- **Feeding coordination beliefs:** upon receiving the empty bowl notification, it does not act directly but triggers a plan to delegate the work to a feeding operator;
- **Medical transport beliefs:** handles requests from the Veterinarian, logically distinguishing between outbound transit toward the treatment room and return transit to the cage of origin.

```

!boot.
+!boot <-
    .mark_bdi_ready.
+!bowl_empty(TaskId, CageId, BowlId) <-
    !ensure_food(TaskId, CageId, BowlId).
+!ensure_food(TaskId, CageId, BowlId) <-
    .request_feeding(TaskId, CageId, BowlId).
+!feeding_completed(TaskId) <-
    .finish_feeding_workflow(TaskId, completed).
+!feeding_failed(TaskId) <-
    .finish_feeding_workflow(TaskId, failed).
+!transport_requested(TaskId, AnimalId, CageId, outbound) <-
    .execute_transport(TaskId, AnimalId, CageId, outbound).
+!transport_requested(TaskId, AnimalId, CageId, return) <-
    .execute_transport(TaskId, AnimalId, CageId, return).

```

- **Initialization:** certifies that the decision engine is ready;
- **Food dispatching:** transforms the hunger belief into an order to send to an operator;
- **Transport execution:** transforms the Veterinarian's request into the physical action of animal movement;
- **Conclusion:** records the final outcome in agent memory.

The Python Executive File: `logistics_agent.py` The Python class `LogisticsAgent` manages the dual operational role of the agent: dispatching feeding tasks and physically moving animals in the field. The internal architecture is organized into five components:

- **Alarms and Food Claim (PerceptionReceiver):** Intercepts the empty bowl notification, reserves the task to avoid duplicates, and passes the belief `bowl_empty` to the BDI layer.
- **Food Dispatching (RequestFeeding):** Selects a `FeedingAgent` via Round-Robin rotation to balance the workload, sends the order via FIPA, and informs the BDI layer when the bowl is full.
- **Medical Request Reception (TransportReceiver):** Receives the transport request sent by the `VeterinaryAgent` and activates the BDI layer with the belief `transport_requested`.
- **Physical Animal Transport (ExecuteTransport):** Picks up the animal and delivers it to its destination, managing both the outbound and return journeys while respecting room semaphores and the constraint of one animal per transit.
- **BDI Binding (add_role_actions):** Associates the actions defined in the `.asl` file (`.request_feeding`, `.execute_transport`, `.finish_feeding_workflow`) with the respective asynchronous SPADE behaviours.

4.4 VeterinaryAgent

4.4.1 Role and Nature of the Agent

The **VeterinaryAgent** is the medical officer of the center, responsible for diagnosis, pharmacological prescription, and the administration of therapies to sick animals. Like the **FeedingAgent** and the **LogisticsAgent**, it adopts a two-layer hybrid architecture: a symbolic deliberative BDI layer and an operational, executive Python/SPADE layer. Unlike the logistics agent, it moves strictly and exclusively between the treatment room and the medical storage, where medicines are collected and animals are treated.

4.4.2 Interaction Interfaces

The agent communicates with two entities:

- **With the EnvironmentAgent:**
 - Receives notifications signaling a sick animal;
 - Validates therapeutic actions: requests the reservation of the clinical task, the collection of the medication from the medical storage, and the administration of treatment to the patient.
- **With the LogisticsAgent:**
 - Sends medical transport requests to have the animal delivered to the treatment room on the outbound journey, and to have it returned to its cage once recovered;
 - Receives task acceptance confirmations and patient delivery notifications from Logistics.

4.4.3 The Complete Operational Cycle

The clinical protocol unfolds across six sequential steps:

1. **Reception:** Receives an alarm from the Environment notifying the presence of a sick animal;
2. **Atomic Task Reservation:** Contacts the Environment to lock the clinical task and prevent multiple doctors from taking charge of the same patient;
3. **Outbound Transport Request:** Orders Logistics to retrieve the animal from its cage and position it on the examination table in the treatment room;
4. **Clinical Therapy:** Once the patient is on the table, accesses the medication storage to collect the prescribed drug and administers the treatment to the animal;

5. **Return Transport Request:** Once therapy is completed, contacts Logistics again to return the recovered animal to its cage;
6. **Closure:** Receives confirmation of the animal's return to the cage and closes the clinical record, updating its internal memory.

4.4.4 Separation of Layers: Beliefs File (.asl) vs. Python File (.py)

The agent is strictly divided between its deliberative protocol and physical execution:

A. The Beliefs File: `src/tamagotchi_wild/bdi/veterinary.asl` It represents the clinical mind of the agent: it paces the stages of care according to the progression of the animal's state:

```
!boot.
+!boot <-
    .mark_bdi_ready.
+sick_animal(TaskId, AnimalId, CageId) <-
    !ensure_treatment(TaskId, AnimalId, CageId).
+!ensure_treatment(TaskId, AnimalId, CageId) <-
    .request_medical_transport(TaskId, AnimalId, CageId, outbound).
+patient_ready(TaskId, AnimalId, CageId) <-
    !treat_patient(TaskId, AnimalId, CageId).
+!treat_patient(TaskId, AnimalId, CageId) <-
    .execute_medical_treatment(TaskId, AnimalId, CageId).
+treatment_succeeded(TaskId, AnimalId, CageId) <-
    !ensure_return(TaskId, AnimalId, CageId).
+!ensure_return(TaskId, AnimalId, CageId) <-
    .request_medical_transport(TaskId, AnimalId, CageId, return).
+animal_returned(TaskId, AnimalId, CageId) <-
    .finish_medical_workflow(TaskId, completed).
+medical_failed(TaskId) <-
    .finish_medical_workflow(TaskId, failed).
```

1. **Initialization:** The initial BDI goal `!boot` activates the AgentSpeak plan in `veterinary.asl`, invoking the custom action `.mark_bdi_ready` which unblocks the synchronization event `bdi_ready` in the Python base class `ProjectBDIAgent`.
2. **Outbound Phase:** The assertion of the belief `sick_animal` generates the goal `!ensure_treatment`; the plan executes the action `.request_medical_transport`, delegating FIPA negotiation with logistics to the SPADE asynchronous behaviour `RequestTransport`.
3. **Treatment Phase:** The arrival belief of the patient `patient_ready` activates the goal `!treat_patient`; the action `.execute_medical_treatment` delegates to the behaviour `ExecuteTreatment` the retrieval of the medication and treatment in the environment under mutual exclusion via the lock `treatment_lock`.

4. **Return Phase:** The completion belief `treatment_succeeded` generates the repatriation goal `!ensure_return`, which reuses the transport custom action and the corresponding behaviour `RequestTransport` to return the animal to its cage.
5. **Conclusion:** The final state beliefs `animal_returned` (success) or `medical_failed` (failure/timeout) trigger the action `.finish_medical_workflow`, recording the formal outcome and notifying task closure by unblocking the Python event `workflow_done`.

The Python Executive File: `veterinary_agent.py` The Python class `VeterinaryAgent` serves as the operational layer: it implements asynchronous network communication (SPADE/XMPP) and environment interaction, translating symbolic BDI plans into concrete actions. The internal architecture is organized into four operational components:

- **Reception and Task Claim** (`PerceptionReceiver`): Intercepts the sick animal, reserves the task to avoid conflicts with other veterinarians, and passes the belief `sick_animal` to the BDI layer.
- **Transport Request** (`RequestTransport`): Requests transport of the animal from `LogisticsAgent`.
- **Medical Treatment** (`ExecuteTreatment`): Retrieves medication, treats the animal (one patient at a time via `treatment_lock`), and notifies the BDI layer that the treatment was successful.
- **BDI Binding** (`add_role_actions`): Acts as a bridge, connecting actions defined in the `.asl` file to the Python code that executes them.

4.5 VisualizationAgent

4.5.1 Role and Nature of the Agent

Acts as a read-only bridge between the SPADE simulation and the NiceGUI dashboard. **It is not a BDI agent** (it makes no decisions and has no `.asl` file): it is a purely reactive SPADE agent that projects the grid state to the screen without modifying the environment. For a comprehensive discussion of the web visualization subsystem, the NiceGUI frontend, and the inter-process communication bridge, refer to Section 7.

4.5.2 Interaction Interfaces

- **EnvironmentAgent:** Receives periodic state snapshots via XMPP under the `cras.visualization` ontology.
- **NiceGUI Dashboard:** Forwards received frames to the GUI process via a local thread-safe queue (`frame_queue`).

4.5.3 The Operational Cycle

1. **Reception:** Intercepts the snapshot emitted by the environment;
2. **Validation:** Deserializes the JSON payload into a `VisualizationUpdate` data model;
3. **Dispatch:** Enqueues the frame into `frame_queue` for graphical rendering.

4.5.4 Python Implementation (`visualization_agent.py`)

Lacking a BDI deliberative layer, it is implemented entirely in a single Python file:

- **VisualizationAgent:** Configures message reception templates for visualization frames and signals initialization with the `ready` event.
- **UpdateReceiver:** A cyclic behaviour that consumes incoming messages and pushes validated frames into the GUI queue.

5 Communication Protocol and Message Contracts (`messaging.py`)

In a distributed multi-agent system based on XMPP, agents do not share memory and cannot invoke each other directly: every coordination occurs through asynchronous messages, structured according to the **FIPA-ACL** standard and formalized in the `messaging.py` module.

5.1 Message Structure

Each message has two distinct layers:

- **FIPA-ACL Header:** Contains the `performative` field (the communicative intent: `request`, `agree`, `refuse`, `inform`, `failure`), the `ontology` (the thematic channel), and the `language` field (always set to `json`).
- **Typed JSON Payload:** Application data is encapsulated in immutable Python dataclasses. When an agent needs to communicate something, the `to_json()` method of the corresponding class automatically generates the payload, which is then inserted into the body of the SPADE message and dispatched over the XMPP network. For example, when the `VeterinaryAgent` orders Logistics to bring an animal to the treatment room, the `TransportRequest` class produces at runtime:

```
{
  "schema_version": 1,
  "task_id": "medical_001",
  "animal_id": "animal_001",
```

```

    "cage_id": "cage_01",
    "direction": "outbound",
    "event": "transport_animal"
}

```

with FIPA header `performative="request"`, `ontology="cras.transport"`, `conversation-id="medical_001"`. If the payload received on the other side does not conform to the expected schema, a `MessageContractError` is raised and the message is discarded before reaching the BDI reasoning layer.

5.2 The 5 System Ontologies

Messages are not funnelled into a single channel: they are separated by topic through five distinct ontologies, defined as constants in `messaging.py`:

```

ENVIRONMENT_ONTOLOGY = "cras.environment"
PERCEPTION_ONTOLOGY = "cras.perception"
FEEDING_ONTOLOGY = "cras.feeding"
TRANSPORT_ONTOLOGY = "cras.transport"
VISUALIZATION_ONTOLOGY = "cras.visualization"

```

At startup, each agent associates its behaviours with a SPADE `Template` that filters incoming messages by ontology. In the `VeterinaryAgent`, for example, only messages carrying the `cras.perception` ontology wake up the `PerceptionReceiver` behaviour:

```

async def setup(self) -> None:
    template = Template()
    template.set_metadata("performative", "inform")
    template.set_metadata("ontology", PERCEPTION_ONTOLOGY)
    self.add_behaviour(self.PerceptionReceiver(), template)

```

The SPADE runtime automatically routes messages to the correct behaviour, with no need for manual `if/else` checks on message content.

5.3 Task Isolation via conversation-id

With 10 animals and 7 operators active in parallel, dozens of messages travel the network simultaneously. To prevent a veterinarian from intercepting a reply intended for a colleague, each task carries a unique identifier (`task_id`) injected as `conversation-id` in the FIPA header. The sending agent then listens for replies using a template bound to that exact code:

```

template = Template()
template.set_metadata("ontology", TRANSPORT_ONTOLOGY)
template.set_metadata("conversation-id", self.task_id)

```

Any message lacking the expected identifier is silently ignored, guaranteeing complete isolation between concurrent tasks.

5.4 The FIPA Sequence in Action

The transport of an animal (`medical_001`) follows three formal acts:

1. **request:** The Veterinarian orders the transport from Logistics (`direction=outbound`);
2. **agree:** The logistics operator formally accepts the task (`transport_accepted`), without having completed it yet;
3. **inform:** Only once the patient has been placed on the examination table does the operator send the completion notification (`patient_ready`), unblocking the physician to proceed with therapy.

6 Concurrency Management and Mutual Exclusion

In a distributed multi-agent system over an XMPP network, agents do not share memory and cannot directly observe each other's state. Each operator reasons exclusively on information received through messages, with no visibility into what colleagues are doing at the same instant. This absence of shared state makes it impossible to prevent conflicts through global variables or classical synchronization mechanisms. As the system scales to multiple agents per role, two categories of conflict emerge: task contention, where multiple agents attempt to execute the same task concurrently, and spatial congestion, where multiple operators simultaneously request access to a resource with limited capacity. To address both categories, the system adopts a centralized arbiter composed of two layers: the **EnvironmentAgent**, which receives operator requests over the XMPP network, and **EnvironmentState** (`environment/state.py`), which contains the pure Python validation logic and to which the **EnvironmentAgent** delegates every decision. Three distinct mechanisms cover the three identified levels of conflict: exclusive task reservation, room access control, and serialization of medical procedures.

6.1 Atomic Task Reservation

Before undertaking any action, each agent is required to formally register its intent with the **EnvironmentAgent** through a **CLAIM.TASK** request. The handling of such a request is delegated to the `_apply_task_claim` method, defined in `environment/state.py`. The first agent to submit the request for a (`task_id`, `phase`) pair (e.g., (`medical_001`, `medical_coordination`)) acquires the lock, recording its own identifier in the internal `_task_claims` structure. Any subsequent agent attempting to claim the same phase receives an `accepted=False` response via the **failure** performative, abandoning the task without retrying. The mechanism also incorporates a consistency check between the requesting agent's role and the claimed phase: an agent with role **FEEDING** cannot acquire

a `medical_coordination` phase, for example. The following is an excerpt from `environment/state.py`:

```
key = (task.id, command.target_id)
owner = self._task_claims.get(key)
if owner is not None:
    return ActionResult(False, f"task phase already claimed by {
        owner}")
self._task_claims[key] = command.actor_id
```

6.2 Room Semaphores

Each operational area of the rescue center admits at most two operators at a time. Access control is implemented in the `_apply_area_acquire` method, defined in `environment/state.py`, which verifies the cardinality of the `_area_occupants[area_id]` set before granting entry.

```
occupants = self._area_occupants[area.id]
capacity = min(area.capacity, 2)
if len(occupants) >= capacity:
    raise InvalidActionError(f"area {area.id} is at capacity")
```

If the room is full, the request is rejected and the agent enters the retry loop defined in the next section. If access is granted instead, the agent's ID is added to `_area_occupants` and its position is updated on the grid. The method then enforces a second constraint: an agent cannot be in two rooms at the same time. This was implemented to prevent further deadlocks. Release occurs through `RELEASE_AREA`: the agent notifies the environment that its task is complete, removing its own ID from `_area_occupants` and thus freeing the slot for any waiting agent.

6.3 Non-Blocking Retry Loop

When an `ACQUIRE_AREA` request is rejected, the agent neither blocks nor terminates execution. The auxiliary function `acquire_area`, defined in `src/tamagotchi_wild/agents/environment_actions.py`, implements an asynchronous polling loop with a deadline:

```
deadline = asyncio.get_running_loop().time() + behaviour.agent.
    timeout_seconds
while True:
    result = await environment_action(behaviour, task_id,
        ActionType.ACQUIRE_AREA, ...)
    retryable = "is at capacity" in result.reason \
        or "is busy with task" in result.reason
    if result.accepted or not retryable:
        return result
    if asyncio.get_running_loop().time() >= deadline:
        return ActionResult(task_id, False,
            f"timeout waiting for {area_id}",
            None)
    await asyncio.sleep(0.05)
```

When a `LogisticsAgent` attempts to enter the Medical Storage but two agents are already working inside, instead of stopping and waiting while occupying the thread, upon receiving `accepted=False` it waits 50 ms and retries. This repeats until one of the agents occupying the room emits `RELEASE_AREA`. If `timeout_seconds` elapse without anyone leaving the room, the task is automatically terminated. The code uses `await asyncio.sleep` rather than a blocking loop because other agents must continue running in the meantime. If the `LogisticsAgent` were to block the thread, other agents could never complete their tasks and emit `RELEASE_AREA` to free the slot, causing a deadlock. With `await`, control is yielded to the event loop, preserving the system.

7 Graphical Visualization Subsystem

Real-time observability of the multi-agent system is provided by a dedicated web dashboard strictly decoupled from the core simulation runtime. The visualization architecture is organized into four complementary components:

7.1 The Web Presentation Framework (NiceGUI)

For the presentation tier, **NiceGUI** was adopted—a Python web framework built on top of FastAPI and Vue.js. This choice was driven by the objective of maintaining the entire codebase within the Python ecosystem, while ensuring:

- **Real-Time Synchronization:** state updates are pushed through bidirectional, persistent **WebSocket** connections, eliminating HTTP polling overhead;
- **Native Vector Rendering:** the interface directly manipulates **SVG** elements, enabling dynamic rendering of the grid floor plan without external JavaScript build toolchains.

7.2 Inter-Process Communication and Decoupling (`gui_bridge.py`)

The asynchronous event loop of SPADE (bound to XMPP network I/O) features concurrency constraints incompatible with the NiceGUI web server runtime. Running both subsystems within the same event loop would induce severe thread contention, where frontend rendering latency or browser disconnections could stall agent asynchronous tasks. To guarantee strict **execution domain isolation**, `gui_bridge.py` establishes an **Inter-Process Communication (IPC)** bridge:

- Executes the multi-agent simulation in a dedicated, isolated process;
- Encapsulates world state snapshots emitted by the `VisualizationAgent` and streams them across an authenticated local socket connection (`multiprocessing.connection`);

- Enforces fault tolerance: client rendering delays or browser reconnections cannot stall state transitions within the multi-agent environment.

7.3 Graphical Presentation Layer (`visualization/nicegui_view.py`)

This module implements the user-facing web frontend, translating state payloads delivered by the bridge into interactive graphical components:

- **Interactive SVG Floor Plan:** geometrically renders the 12×8 grid, color-coding the four operational areas and static objects (bowls, medical examination stations);
- **Dynamic Entity Tracking:** tracks operator coordinates in real time and projects animal clinical status and species via dedicated icons;
- **Event and Diagnostic Timeline:** chronologically logs physical actions accepted or rejected by the environment;
- **Metrics and Control Panel:** exposes playback speed selection, simulation visualization pause/resume controls, and aggregated counters for patient health and inventory stocks.

7.4 Role of the VisualizationAgent within the Pipeline

Within the SPADE multi-agent network, the **VisualizationAgent** (introduced in Section 4.5) operates strictly as a passive sensor: devoid of a BDI deliberative engine or `.asl` plans, it listens for periodic broadcast messages emitted by the **EnvironmentAgent** under the `cras.visualization` ontology, validates schema compliance (`VisualizationUpdate`), and delegates frame forwarding to the IPC bridge.

Methodological Note on Tooling The architectural design of the multi-process asynchronous bridge and the dynamic vector SVG layout were developed with the assistance of Large Language Models (LLMs) used as a pair-programming and rapid-prototyping tool for the interface scaffolding.